Inteligencia Artificial

Optimización del problema PSP-UAV mediante algoritmos genéticos y simulación de urgencia acumulada

Benjamín Renzo Ferrada Larach

25/11/2025

Evaluación

Mejoras 2da Entrega (10 %):	_____
Código Fuente (10 %):	_____
Representación (5 %):	_____
Descripción del algoritmo (15 %):	_____
Experimentos (10 %):	_____
Resultados (40 %):	_____
Conclusiones (10 %):	_____
Nota Final (100):	_____

Resumen

En este informe estudiamos el problema PSP-UAV, que consiste en planificar las rutas de una flota de vehículos aéreos no tripulados sobre una grilla con celdas de distinta urgencia, con el objetivo de minimizar la urgencia acumulada a lo largo del horizonte de planificación. Para resolverlo implementamos un algoritmo genético inspirado en el enfoque *Multi-Mission Learning Workflow with Genetic Algorithms* (MMLW-GA), adaptando la función objetivo a la utilizada en la asignatura, la cual se basa directamente en la urgencia acumulada por celda y por unidad de tiempo. Evaluamos el desempeño del algoritmo en seis instancias de distinto tamaño y estructura, variando parámetros como el tamaño de la población y el número de generaciones, y analizamos las rutas generadas mediante visualizaciones y animaciones. En general observamos que este enfoque evolutivo logra reducir significativamente la urgencia acumulada, destacando la importancia que tiene la correcta configuración de los parámetros del algoritmo.

1. Introducción

En este informe describimos la implementación de un algoritmo genético (AG) orientado a resolver una variante del problema PSP-UAV, donde se debe planificar el desplazamiento de múltiples drones sobre una grilla discreta con celdas que presentan distintos niveles de urgencia. El objetivo principal es diseñar rutas que reduzcan la suma total de urgencias acumuladas durante un horizonte temporal finito.

Este tipo de problema aparece en aplicaciones como vigilancia, monitoreo de infraestructura crítica, respuesta ante emergencias y tareas de inspección periódica. En estos contextos, la capacidad de identificar zonas prioritarias y coordinar múltiples drones es clave para ofrecer un buen nivel de servicio.

El documento se estructura de la siguiente manera. En la Sección 2 definimos formalmente el problema. En la Sección 3 resumimos brevemente el estado del arte en planificación de rutas

para UAVs y en el uso de algoritmos genéticos. La Sección 4 presenta un modelo matemático de referencia escrito en LaTeX. En la Sección 5 describimos la representación interna del problema y del cromosoma. En la Sección 6 detallamos la implementación del algoritmo genético. En la Sección 7 explicamos el diseño experimental y las instancias utilizadas. En la Sección 8 analizamos los resultados obtenidos. Finalmente, en la Sección 9 entregamos las conclusiones y posibles líneas de trabajo futuro.

2. Definición del Problema

Consideramos una grilla rectangular de tamaño $R \times C$, con celdas indexadas por (r, c), donde $r \in \{0, \ldots, R-1\}$ y $c \in \{0, \ldots, C-1\}$. Cada celda puede ser:

- Una celda de *urgencia*, con peso $w_{r,c} > 0$.
- Un obstáculo (celda inaccesible).
- Una celda sin urgencia ($w_{r,c} = 0$).

Disponemos de una flota de K drones (UAVs) que avanzan en pasos discretos de tiempo $t = 0, 1, \ldots, T-1$. En cada tick, cada dron puede:

- Permanecer en su celda actual, o
- Moverse a una celda adyacente (vecindad de 8) que no sea obstáculo.

La urgencia de una celda crece linealmente en el tiempo mientras no es visitada, y se reinicia a cero cuando al menos un dron la visita. Sea $U_{r,c}(t)$ la urgencia acumulada de la celda (r, c) en el tick t. Su dinámica es:

$$U_{r,c}(0) = 0, \tag{1}$$

$$U_{r,c}(t+1) = \begin{cases} 0, & \text{si algún dron visita } (r,c) \text{ en el tick } t, \\ U_{r,c}(t) + w_{r,c}, & \text{en caso contrario.} \end{cases} \tag{2}$$

El objetivo es diseñar rutas que minimicen la suma total de urgencias acumuladas:

$$F = \sum_{t=0}^{T-1} \sum_{r=0}^{R-1} \sum_{c=0}^{C-1} U_{r,c}(t). \tag{3}$$

Las restricciones principales son:

- Movimiento adyacente.
- Prohibición de entrar a obstáculos.
- Los drones deben permanecer dentro de la grilla.
- El número de drones K es fijo para la instancia.

Este problema se relaciona con variantes de vigilancia persistente, cobertura periódica y planificación de rutas con prioridades dinámicas.

3. Estado del Arte

La planificación de rutas para UAVs en grillas con prioridades espaciales ha sido estudiada desde distintas perspectivas. Muchos trabajos abordan variantes del Vehicle Routing Problem (VRP) con múltiples vehículos, restricciones temporales y prioridades, así como problemas propios de vigilancia persistente.

Las metaheurísticas —incluyendo algoritmos genéticos— han demostrado buena capacidad para resolver problemas donde los modelos exactos escalan mal. En particular, en [3] se presenta el enfoque *Multi-Mission Learning Workflow with Genetic Algorithms* (MMLW-GA), el cual usa un GA para aprender patrones de visita sobre un conjunto de *sites* y reutilizarlos en distintas misiones. En nuestro caso utilizamos ideas de esta representación, pero reemplazamos la función objetivo original por la urgencia acumulada definida en esta asignatura.

Otros trabajos relevantes incluyen enfoques basados en colonia de hormigas, recocido simulado, búsqueda tabú y métodos híbridos [1, 2], los cuales destacan la importancia de diseñar representaciones y operadores que mantengan soluciones factibles. Estos lineamientos también guiaron el diseño de nuestro algoritmo.

4. Modelo Matemático

En esta sección presentamos un modelo matemático para el problema PSP-UAV en términos de variables binarias. Sea:

- R, C número de filas y columnas.
- T horizonte temporal (número de ticks).
- K número de drones.
- $w_{r,c}$ urgencia base de la celda (r, c).
- $\mathcal{N}(r, c)$ conjunto de celdas adyacentes (en 8 vecinos) a (r, c), incluyendo la propia celda.

Variables de decisión:

$$x_{k,r,c,t} = \begin{cases} 1 & \text{si el dron } k \text{ está en la celda } (r,c) \text{ en el tiempo } t, \\ 0 & \text{en caso contrario,} \end{cases} \tag{4}$$

para todo $k \in \{1, \ldots, K\}$, $r \in \{0, \ldots, R-1\}$, $c \in \{0, \ldots, C-1\}$ y $t \in \{0, \ldots, T-1\}$.

Variable auxiliar de visita:

$$v_{r,c,t} = \begin{cases} 1 & \text{si algún dron visita la celda } (r,c) \text{ en el tiempo } t, \\ 0 & \text{en caso contrario.} \end{cases} \tag{5}$$

Urgencia acumulada en cada celda y tiempo:

$$u_{r,c,t} \geq 0, \quad \forall r, c, t. \tag{6}$$

La relación entre visitas y drones es:

$$v_{r,c,t} \geq x_{k,r,c,t}, \quad \forall k, r, c, t. \tag{7}$$

La dinámica de la urgencia acumulada se modela como:

$$u_{r,c,0} = 0, \quad \forall r, c, \tag{8}$$

$$u_{r,c,t+1} \geq u_{r,c,t} + w_{r,c} - M\, v_{r,c,t}, \quad \forall r, c,\ t = 0, \ldots, T-2, \tag{9}$$

donde M es una constante suficientemente grande.

Cada dron ocupa exactamente una celda en cada tiempo:

$$\sum_{r,c} x_{k,r,c,t} = 1, \quad \forall k, t. \tag{10}$$

Restricción de movimiento adyacente:

$$x_{k,r,c,t+1} \leq \sum_{(r',c') \in \mathcal{N}(r,c)} x_{k,r',c',t}, \quad \forall k, r, c, \ t = 0, \ldots, T-2. \tag{11}$$

Prohibición de obstáculos:

$$x_{k,r,c,t} = 0, \quad \forall k, t \text{ y celdas } (r,c) \text{ que son obstáculo.} \tag{12}$$

La función objetivo coincide con (3):

$$\text{mín } F = \sum_{t=0}^{T-1} \sum_{r=0}^{R-1} \sum_{c=0}^{C-1} u_{r,c,t}. \tag{13}$$

Este modelo sirve como referencia teórica; en la práctica, el algoritmo genético trabaja con una representación más compacta, descrita en la siguiente sección.

5. Representación

Para representar una solución utilizamos un esquema inspirado en MMLW-GA [3]. En vez de codificar directamente la posición de cada dron en cada tick, definimos un conjunto de *sites* $S = \{1, \ldots, m\}$, correspondientes a celdas con urgencia positiva. A partir de estos sites construimos un grafo completo ponderado por distancias (matriz D), donde $D_{i,j}$ representa el costo de desplazarse desde i a j mediante movimiento válido en la grilla.

Cada individuo en el GA se representa mediante un *cromosoma*, que corresponde a un *walk* sobre los sites:

- El cromosoma es un vector de longitud fija N.
- La función `decode` transforma el cromosoma en un walk W.
- A partir del walk construimos un *kernel* que detalla la secuencia completa de celdas recorridas.

El kernel es una secuencia:

$$K = \big((r_0, c_0), \ldots, (r_{L-1}, c_{L-1})\big),$$

y mediante él asignamos rutas a los K drones:

$$R^d = \big((r_0^d, c_0^d), \ldots, (r_{T-1}^d, c_{T-1}^d)\big).$$

Esta representación separa la lógica del GA del movimiento de bajo nivel en la grilla, lo que permite implementar operadores genéticos simples sobre enteros.

6. Descripción del algoritmo

El algoritmo implementado es un algoritmo genético estándar adaptado al problema PSP-UAV. A diferencia del esquema del paper original, la función objetivo utilizada en la evaluación de cada individuo corresponde exactamente a la urgencia acumulada definida en la Sección 4, calculada mediante una simulación explícita de las rutas de los drones.

6.1. Estructura general

El pseudocódigo de alto nivel se muestra en el Algoritmo 1.

Algorithm 1 Algoritmo genético para PSP-UAV

```
Generar población inicial de P cromosomas aleatorios.
for generación g = 1 hasta G do
    for cada cromosoma c en la población do
        Decodificar c en un walk W sobre sites.
        Construir el kernel K a partir de W.
        Asignar rutas R^1, ..., R^K a los drones usando K.
        Simular la evolución de urgencias en la grilla durante T ticks.
        Calcular la urgencia acumulada F(c) (función objetivo).
        Aplicar penalizaciones por movimientos inválidos u otras violaciones.
    end for
    Seleccionar cromosomas para reproducción (elitismo + truncation).
    Aplicar cruce con probabilidad p_cross.
    Aplicar mutación con probabilidad p_mut.
    Formar la nueva población.
end for
Devolver el mejor cromosoma encontrado y sus rutas asociadas.
```

6.2. Operadores genéticos

Selección. Utilizamos una selección por truncamiento con elitismo: los individuos se ordenan de menor a mayor fitness (urgencia acumulada) y se preserva el mejor cromosoma en la siguiente generación. El resto de la población se rellena seleccionando parejas de padres de la parte superior del ranking.

Cruce. El operador de cruce intercambia sufijos de dos cromosomas con probabilidad p_{cross}, eligiendo un punto de corte uniforme al azar. Este operador respeta la longitud fija del cromosoma y mezcla subsecuencias de sites.

Mutación. La mutación se implementa mediante tres tipos de modificaciones sobre los genes:

- Modificación de índices de *swap*.
- Reasignación de nodos adicionales.
- Cambios en posiciones de inserción.

Cada gen tiene una probabilidad p_{mut} de ser modificado, y los nuevos valores se eligen uniformemente en rangos válidos. Esto permite explorar diferentes walk sobre los sites.

6.3. Función de fitness y penalizaciones

La función de fitness $F(c)$ de un cromosoma c corresponde a la urgencia acumulada total obtenida al simular las rutas de los drones durante T ticks, partiendo del kernel construido a partir de c. Además, incluimos penalizaciones que aumentan el valor de la función objetivo cuando se detectan situaciones no deseadas:

- Pasos fuera de la grilla.
- Paso por celdas marcadas como obstáculos.

- Movimientos no adyacentes (que violan la vecindad de 8 vecinos).
- Drones completamente ociosos (que no se mueven).
- Celdas con urgencia positiva que nunca son visitadas.

El fitness final tiene la forma:

$$F(c) = \text{urgencia acumulada} + \text{penalizaciones},$$

y el AG busca minimizar $F(c)$.

7. Experimentos

7.1. Instancias utilizadas

Utilizamos seis instancias de prueba provistas en la asignatura, de diferente tamaño y estructura:

- **PSP-UAV_01_a**: grilla pequeña, con 10x10 celdas, aproximadamente 70 celdas con urgencia positiva y varios obstáculos dispersos.
- **PSP-UAV_01_b**: instancia de tamaño similar a 01_a, pero con distinta distribución de urgencias y obstáculos.
- **PSP-UAV_02_a**: grilla de tamaño medio, con mayor número de celdas urgentes y patrones de urgencia más concentrados.
- **PSP-UAV_02_b**: variante de 02_a con distinta distribución y mayor densidad de obstáculos.
- **PSP-UAV_03_a**: instancia grande, que representa un escenario más complejo de cobertura.
- **PSP-UAV_03_b**: instancia grande alternativa para evaluar la robustez del algoritmo.

En todas las instancias, las bases tienen urgencia cero y se permite que despeguen varios drones desde una misma base (cuando las instancias las incluyen explícitamente).

7.2. Configuraciones del algoritmo

El algoritmo se ejecutó variando los parámetros de población y generaciones, mientras que otros parámetros se mantuvieron fijos. Las configuraciones principales fueron:

- Tamaño de población $P \in \{50, 100, 200\}$.
- Número de generaciones $G \in \{20, 40, 60\}$.
- Probabilidad de cruce $p_{\text{cross}} \in \{0{,}7, 0{,}9\}$.
- Probabilidad de mutación $p_{\text{mut}} = 0{,}01$.
- Horizonte temporal $T = 50$ ticks.
- Número de drones K según la instancia (por ejemplo, $K = 5$ para PSP-UAV_01_a).

Para cada combinación de parámetros realizamos varias ejecuciones con distintas semillas aleatorias, lo que permite calcular promedios y desviaciones estándar de la urgencia acumulada y del tiempo de ejecución. Las corridas se automatizaron mediante un script `sweep.py`, que genera archivos CSV con los resultados y gráficos resumen.

7.3. Entorno de experimentación

Las pruebas se realizaron en un computador de escritorio con sistema operativo Windows 11 Pro 25H2, procesador AMD Ryzen 5 7600X a 4,70 GHz y 32 GB de RAM DDR5 a 6000 MT/s. El código se implementó en `C++17` y se compiló con `g++` desde el subsistema de Windows `WSL2` de *Ubuntu 24.04*. Las visualizaciones y animaciones se generaron en Python utilizando `matplotlib` y `numpy`.

8. Resultados

En esta sección presentamos los resultados obtenidos mediante el AG para las seis instancias, destacando el efecto de los parámetros de población y número de generaciones sobre la calidad de las soluciones (urgencia acumulada) y el tiempo de cómputo.

8.1. Convergencia del algoritmo

La Figura 1 muestra, a modo de ejemplo, la evolución del mejor valor de la función objetivo por generación para una ejecución representativa en la instancia PSP-UAV_03_b.

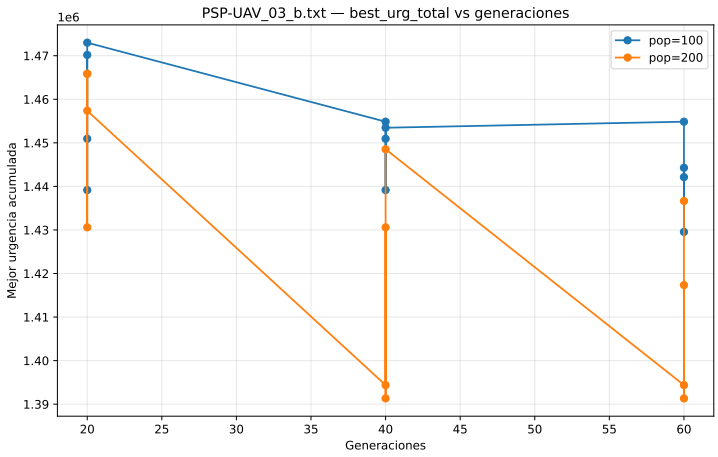


Figura 1: Convergencia del algoritmo genético en la instancia PSP-UAV_03_b.

En general, observamos una disminución rápida de la urgencia acumulada durante las primeras generaciones, seguida de una fase de refinamiento más lenta. Al incrementar el número de generaciones a $G = 40$ o $G = 60$, la mejora adicional existe pero es más bien marginal en comparación con el costo computacional añadido.

8.2. Comparación entre configuraciones

La Tabla 1 muestra un ejemplo de estructura para resumir resultados promedio (sobre varias semillas) en la instancia PSP-UAV_03_b, variando población y generaciones. Los valores numéricos deben extraerse de los CSV generados por `sweep.py` y rellenarse en la versión final del informe.

Cuadro 1: Resultados experimentales en PSP-UAV_03_b variando población y generaciones.

Población	Generaciones	p_{cross}	p_{mut}	Mejor urgencia	Tiempo [s]
200	40	0,7	0,01	1 391 326	92.517
200	60	0,7	0,01	1 391 326	134.590
200	60	0,7	0,01	1 394 380	202.293
200	40	0,7	0,01	1 394 380	138.430
200	60	0,7	0,01	1 417 346	202.610
100	60	0,7	0,01	1 429 552	67.328
200	20	0,7	0,01	1 430 598	70.002
200	40	0,7	0,01	1 430 598	136.422
200	60	0,7	0,01	1 436 647	134.465
100	40	0,7	0,01	1 439 153	45.839

Tablas análogas pueden construirse para las otras instancias PSP-UAV_01_b, 02_a, 02_b, 03_a y 03_b, permitiendo comparar la escalabilidad del algoritmo.

El impacto mostrado por los tamaños de las poblaciones en el tiempo de ejecución total, se muestran en la Figura 2.

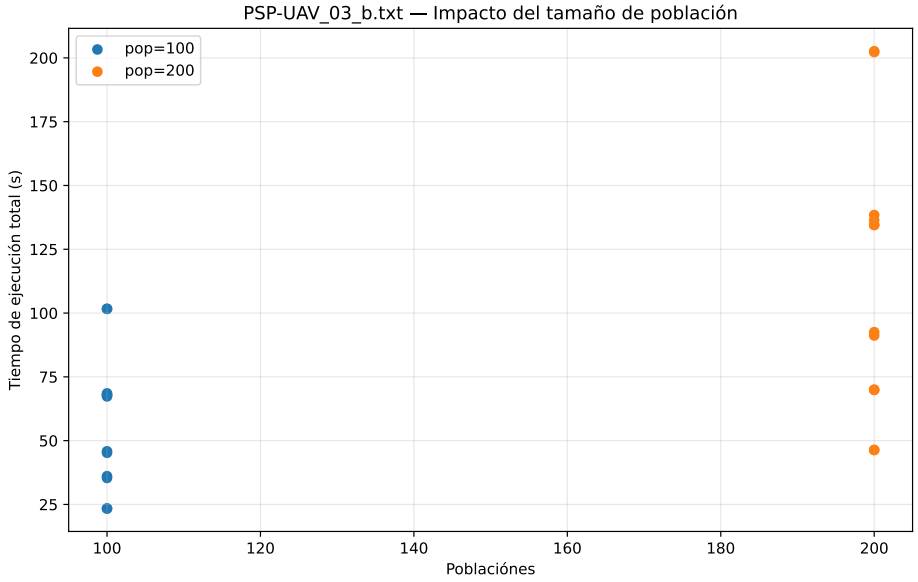


Figura 2: Impacto del tamaño de la población en la instancia PSP-UAV_03_b.

Por último, los tiempos comparativos entre cuanto se demoraba el GA, la simulación y con esto el total, se muestran en la Figura 3.

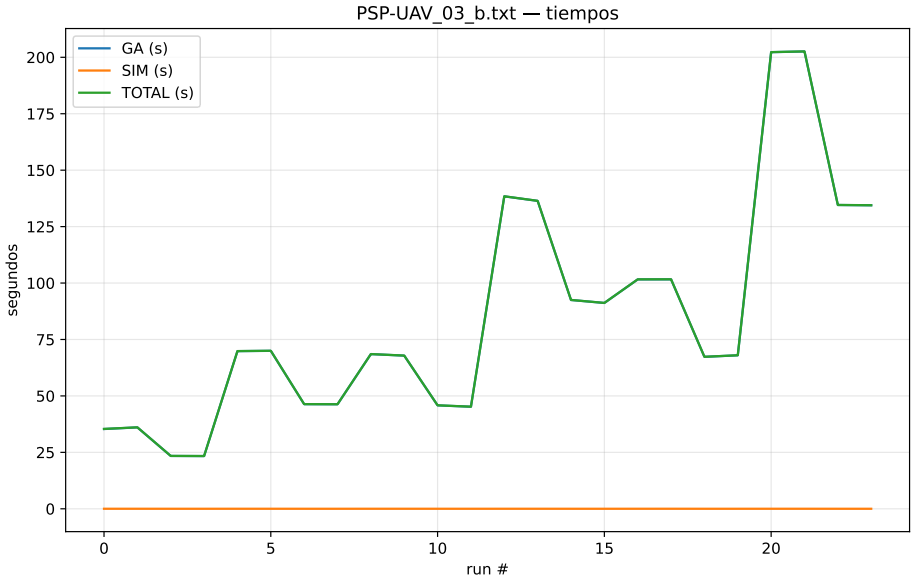


Figura 3: Tiempos totales de ejecución en la instancia PSP-UAV_03_b.

8.3. Visualización de rutas y cobertura

Una ventaja de la representación implementada es que permite visualizar fácilmente las rutas de los drones y la cobertura de la grilla. En la Figura 4 se muestra el grid combinado para una ejecución en la instancia PSP-UAV_03_b, incluyendo sites de urgencia, obstáculos, kernel y rutas de cada dron.

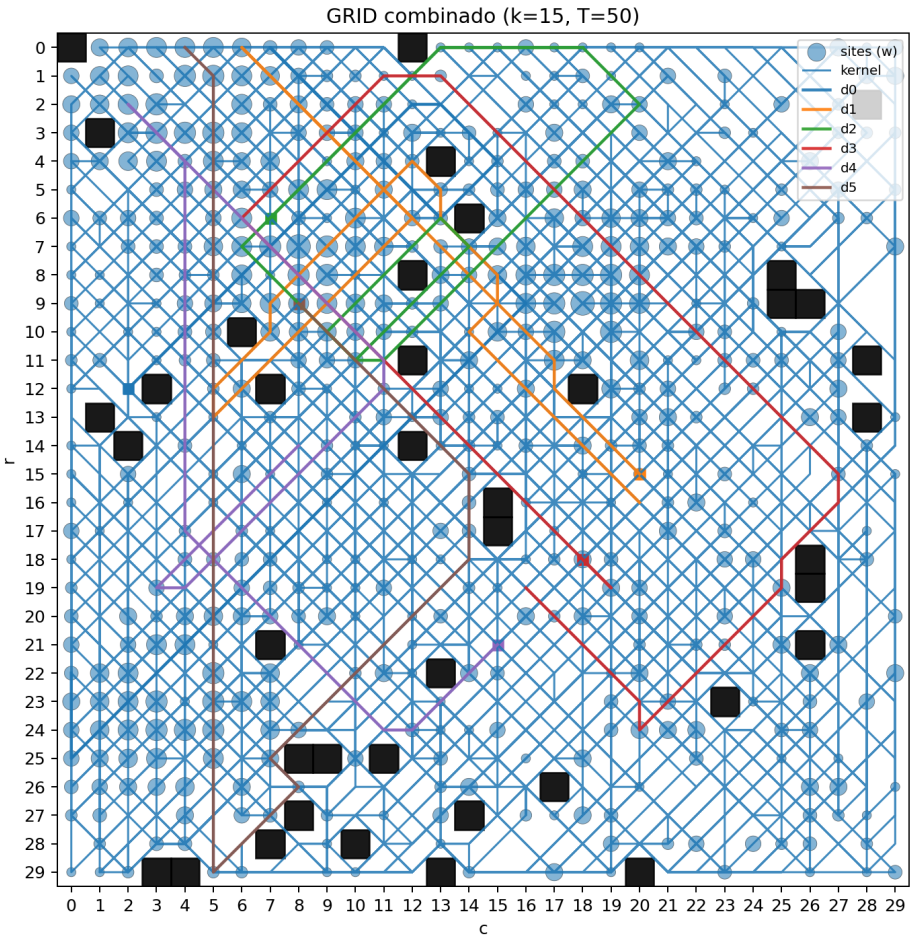


Figura 4: Grid combinado para la instancia PSP-UAV_03_b: sites con urgencia, obstáculos, kernel y rutas de los drones.

Los *heatmaps* de visitas y de frecuencia normalizada (Figuras 5 y 6) muestran cómo los drones concentran su actividad en zonas con mayor urgencia, mientras que las celdas de menor urgencia o sin urgencia reciben menos visitas.

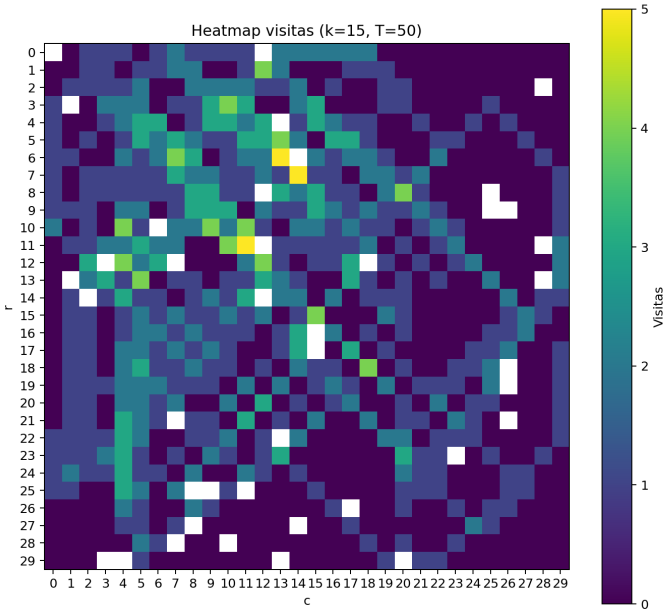


Figura 5: Heatmap de visitas para PSP-UAV_03_b.

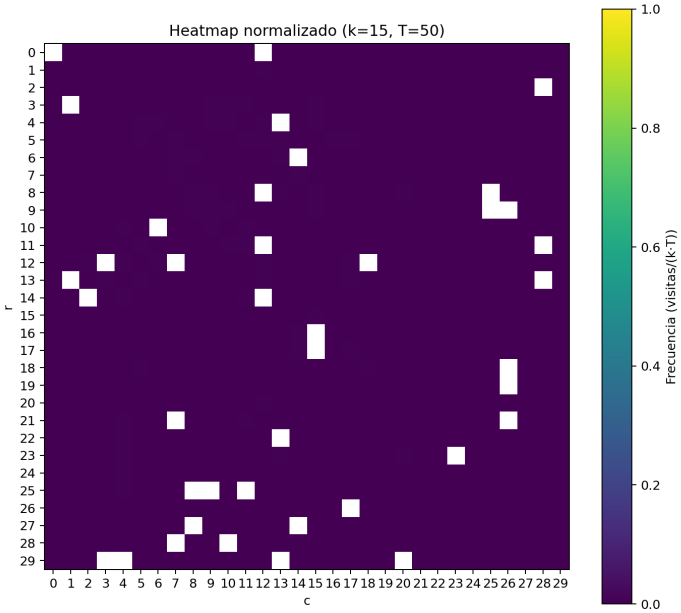


Figura 6: Heatmap normalizado de visitas para PSP-UAV_03_b.

8.4. Animación de las rutas

A partir de los *snapshots* JSON generados por el programa principal, creamos una animación que muestra la evolución de las rutas tick a tick. Para ello exportamos una secuencia de frames numerados `pspuav_PSP-UAV_03_b_g20_p100_n3_s12345_snap_live_frame_000.png`, `...001.png`, etc. En el PDF podemos incluir una animación usando el paquete `animate`, como se ilustra a continuación:

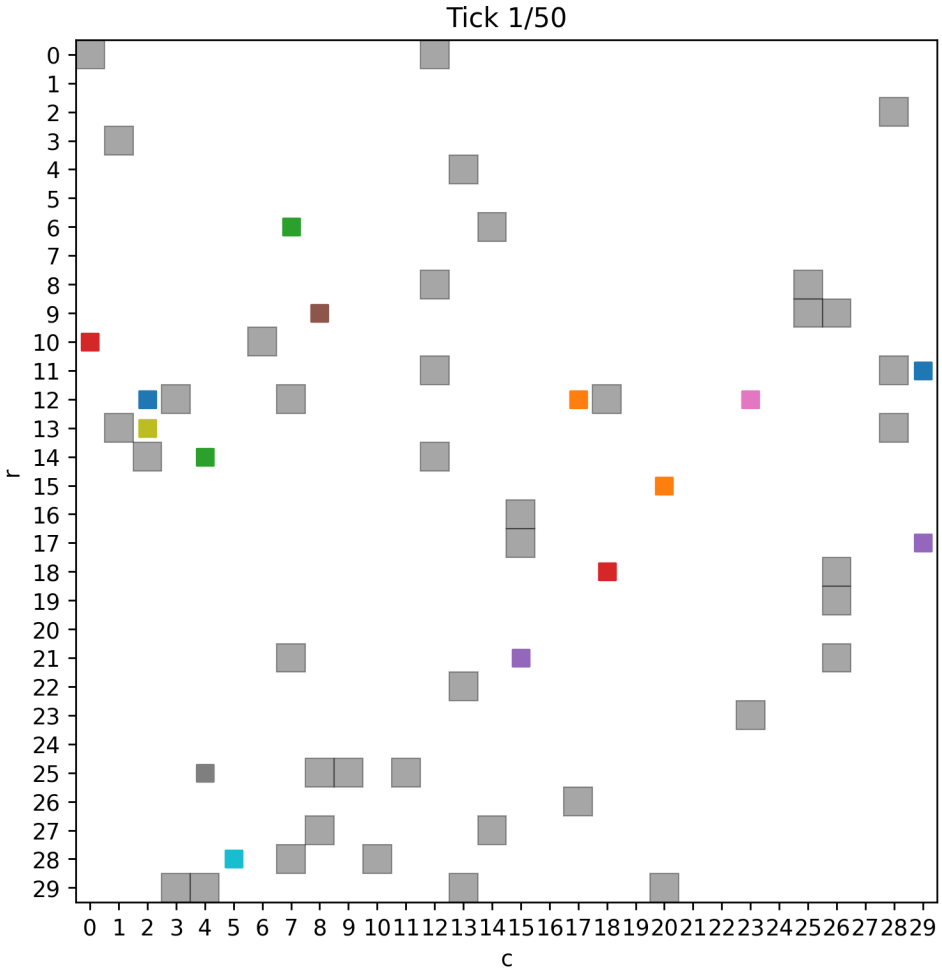


Figura 7: Animación de rutas PSP-UAV_03_b (alta resolución).

En visores que no soporten animación, también es posible mostrar solo algunos frames representativos como figuras estáticas.

9. Conclusiones

En este trabajo implementamos un algoritmo genético para abordar el problema PSP-UAV, adaptando ideas del enfoque MMLW-GA [3] pero empleando una función objetivo basada en urgencia acumulada. La representación mediante un walk sobre sites y un kernel permitió manejar la complejidad de la grilla sin complicar la estructura del cromosoma.

Los experimentos, realizados sobre seis instancias de distinto tamaño, muestran que el GA logra disminuir considerablemente la urgencia acumulada en pocas generaciones. También se observa que parámetros como el tamaño de la población y el número de generaciones influyen directamente en la calidad de las soluciones y en el costo computacional. En particular, las mejoras más significativas ocurren en las primeras generaciones, mientras que incrementos posteriores producen beneficios marginales.

Entre las limitaciones del método destaca la necesidad de calibrar parámetros y el costo elevado de la función fitness, que requiere una simulación completa. Como continuación natural de este trabajo, podrían explorarse operadores más especializados, búsqueda local híbrida o nuevas restricciones inspiradas en aplicaciones reales, como límites energéticos o zonas dinámicamente prohibidas.

Referencias

[1] Patrick Prosser. Hybrid algorithms for the vehicle routing problem with time windows. In *European Conference on Artificial Intelligence*, pages 187–191, 1993.

[2] Pratap Tokekar, James V. Hook, Ashwin Mulla, and Volkan Isler. Multi-uav coordination and coverage: A survey. *Autonomous Robots*, 44(4):567–601, 2020.

[3] H. Zarrouk, M. Seddik, R. Ramadan, and A. Al-Yahmadi. Multi-mission learning workflow with genetic algorithms. *Applied Intelligence*, 53(12):17243–17266, 2023.